

Lernskript: Records & Wither Pattern

Ein Java Record ist eine spezielle Klasse für unveränderliche Datenobjekte. Der Compiler generiert automatisch Konstruktor, Accessor-Methoden, `equals()`, `hashCode()` und `toString()` — ohne eine einzige Zeile Boilerplate-Code.

Teil 1: Die grundlegende Analogie - Das "Warum"

- **Konzept:** Stell dir eine gedruckte Setliste vor. Sie ist unveränderlich — sobald sie gedruckt ist, kann niemand sie heimlich ändern. Wenn die Band eine Änderung will, druckt der Tour Manager eine **neue** Setliste — die alte bleibt wie sie ist.

Das ist **Immutability** (Unveränderlichkeit). Ein Java Record ist diese gedruckte Setliste: einmal erstellt, nie verändert.

- **Das Problem mit normalen Klassen:**

- Eine normale Klasse mit Settern erlaubt es, den Zustand eines Objekts jederzeit zu verändern.
- In Multi-Threading-Szenarien kann das zu inkonsistenten Zuständen führen.
- Der Compiler erzwingt keine Unveränderlichkeit — das muss der Entwickler selbst diszipliniert umsetzen.

- **Die Lösung mit Records:**

- Der Compiler erzwingt Unveränderlichkeit automatisch — keine Setter möglich.
 - Thread-sicher by design: zwei Threads können dasselbe Record-Objekt lesen, ohne sich gegenseitig zu stören.
-

Teil 2: Praktische Anwendungsfälle - Das "Wofür"

Use Case 1: Ein einfacher Record

- **Zweck:** Unveränderliches Datenobjekt mit minimalem Code.
- **Code-Beispiel:**

```
public record TaskDto(Long id, String description, boolean completed) {}
```

- **Was der Compiler automatisch generiert:**

- Einen Konstruktor mit allen drei Parametern
- Accessor-Methoden `id()`, `description()`, `completed()` (kein `get`-Präfix!)
- `equals()`, `hashCode()`, `toString()` basierend auf allen Feldern

Use Case 2: Compact Constructor — Validierung & defensive Kopie

- **Zweck:** Validierung der Eingabedaten beim Erstellen eines Records, ohne einen vollständigen Konstruktor zu schreiben.
- **Code-Beispiel:**

```
public record PoECharacter(String name, String characterClass,
                          int level, String buildType,
                          Map<Attribute, Integer> stats) {
    public PoECharacter {
        // Defensive copy – verhindert, dass externe Referenzen die Map verändern
        stats = Map.copyOf(stats);
    }
}
```

- **Wichtig:** Im Compact Constructor werden die Parameter direkt zugewiesen — kein `this.x = x` nötig. Der Compiler erledigt das automatisch.

Use Case 3: Static Factory Method — kontrollierte Erstellung

- **Zweck:** Statt alle Felder manuell zu befüllen, bietet eine statische Fabrikmethode einen bequemen Einstiegspunkt mit Standardwerten.
- **Code-Beispiel:**

```
public static PoECharacter of(String name, String characterClass,
                              int level, String buildType) {
    Map<Attribute, Integer> defaultStats = new HashMap<>();
    defaultStats.put(Attribute.STRENGTH, 10);
    defaultStats.put(Attribute.DEXTERITY, 10);
    defaultStats.put(Attribute.INTELLIGENCE, 10);
    return new PoECharacter(name, characterClass, level, buildType, defaultStats);
}
```

- **Aufruf:** `PoECharacter.of("Aura Bot", "Witch", 1, "Support")`

Use Case 4: Wither Pattern — kontrollierte Zustandsänderung

- **Zweck:** Da Records keine Setter haben, ermöglicht das Wither Pattern trotzdem Zustandsänderungen — durch Rückgabe eines **neuen** Record-Objekts mit dem geänderten Wert.
- **Code-Beispiel:**

```
public PoECharacter withLevel(int newLevel) {
    return new PoECharacter(name, characterClass, newLevel, buildType, stats);
}

public PoECharacter withStat(Attribute attr, int value) {
    Map<Attribute, Integer> newStats = new HashMap<>(stats);
    newStats.put(attr, value);
}
```

```
return new PoECharacter(name, characterClass, level, buildType, newStats);  
}
```

- **Aufruf mit Method Chaining:**

```
PoECharacter character = PoECharacter.of("Storm Caller", "Witch", 1, "Caster")  
    .withLevel(50)  
    .withStat(Attribute.INTELLIGENCE, 150)  
    .withStat(Attribute.DEXTERITY, 40);
```

Teil 3: Vertiefung (JavaMasta's Profi-Tipps)

- **Warum Records keine JPA-Entities sein können:** JPA (Hibernate) braucht zwingend einen No-Argument-Konstruktor und Setter, um Objekte nach dem Laden aus der Datenbank zu befüllen. Records haben keinen No-Arg-Konstruktor und keine Setter — sie sind für JPA unbrauchbar. Entities müssen klassische mutable Klassen bleiben.
- **Records vs. Lombok @Value :** Lombok's `@Value` erzeugt ebenfalls unveränderliche Klassen. Der Unterschied: Records sind ein offizielles Java-Sprachfeature (seit Java 16 stabil), `@Value` ist eine externe Bibliothek. Für neue Projekte sind Records die bevorzugte Lösung.
- **Thread-Safety durch Immutability:** Da ein Record-Objekt nach der Erstellung nie verändert werden kann, können mehrere Threads dasselbe Objekt gleichzeitig lesen — ohne Synchronisation, ohne Race Conditions. Das ist einer der grössten Vorteile von Immutability in modernen Anwendungen.
- **Map.copyOf() ist entscheidend:** Ohne die defensive Kopie im Compact Constructor könnte der Aufrufer die originale Map nach der Übergabe noch verändern — und damit indirekt den Record-Inhalt. `Map.copyOf()` verhindert das: die interne Map ist unveränderlich und von der Außenwelt entkoppelt.